	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorio de docencia

Laboratorio de Computación Salas A y B

Profesor(a): M.C. René Adrián Dávila Pérez

Asignatura: Programación orientada a objetos

Grupo: 07

No de práctica(s): 10

Integrante(s): 424045919 | 323087067 | 426060033 | 426059961 | 323297930

No de lista o brigada: 1

Semestre: 2027 - 1

Fecha de entrega: 06 de noviembre de 2026

Observaciones:

Calificación:

ÍNDICE

1 Introducción **2**

1.1 Planteamiento del Problema 2

1.2 Motivación 2

1.3 Objetivos 2

1.3.1 Objetivo General 2

1.3.2 Objetivos Específicos 2

2 Marco Teórico **2**

3 Desarrollo **3**

3.1 Descripción de la Implementación (Captura en Superclases por Herencia) 3

3.2 Consultas a IA Generativa 4

3.3 Diagramas UML 5

3.4 Pruebas de Escritorio 8

4 Resultados **8**

5 Conclusiones **8**

6 Referencias **9**

1. INTRODUCCIÓN

1.1. Planteamiento del Problema

Durante el ciclo de ejecución de las aplicaciones orientadas a objetos, la ocurrencia de eventos anómalos o condiciones imprevistas interrumpe el flujo normal de instrucciones. Las fallas dentro del software pueden clasificarse en sintácticas, semánticas y de ejecución. Cuando se produce una condición de error no controlada, como la validación fallida de un parámetro o un estado de saldo insuficiente, la máquina virtual finaliza de forma abrupta el hilo de ejecución. El problema asignado al Equipo 1 consiste en implementar el control de fallos mediante la captura en superclases (Exception), utilizando la jerarquía de herencia para interceptar polimórficamente excepciones de subclases específicas (como IllegalArgumentException) lanzadas durante operaciones de procesamiento de pagos.

1.2. Motivación

El diseño de software de calidad industrial exige tolerancia a fallos y la capacidad de recuperarse ante estados anómalos sin interrumpir la experiencia del usuario. La implementación rigurosa de mecanismos de manejo de excepciones mediante bloques try-catch-finally, la propagación explícita con la cláusula throws y el uso del polimorfismo sobre superclases de excepciones constituyen pilares esenciales para construir aplicaciones robustas. Comprender la diferenciación

técnica entre errores no recuperables de la máquina virtual y excepciones controladas por el desarrollador, así como la distinción entre excepciones verificadas y no verificadas, permite estructurar códigos limpios, mantenibles y conformes a los estándares de ingeniería de software exigidos en la Facultad de Ingeniería de la UNAM.

1.3. Objetivos

1.3.1. Objetivo General

Identificar bloques de código propensos a generar errores y aplicar técnicas adecuadas para el manejo de situaciones excepcionales en tiempo de ejecución.

1.3.2. Objetivos Específicos

- Distinguir conceptual y prácticamente la diferencia entre errores del entorno de ejecución (Error) y situaciones excepcionales controlables (Exception) dentro de la jerarquía estándar de Java.
- Estructurar el control de fallos mediante el mecanismo de captura en superclases (Catching Exceptions via Superclasses), aplicando polimorfismo para gestionar múltiples subclases derivantes mediante un único bloque de captura base.
- Analizar las clases de la jerarquía de errores en Java y las excepciones más comunes en interfaces gráficas (GUI) mediante consultas asistidas por Inteligencia Artificial Generativa (Ejercicios 1.a y 1.b).

2. MARCO TEÓRICO

El manejo de excepciones en la Programación Orientada a Objetos representa la arquitectura formal para separar la lógica de negocio principal del código encargado de gestionar situaciones anómalas en tiempo de ejecución [1]. En la plataforma Java, la jerarquía de manejo de errores está encabe-

zada por la clase base java.lang.Throwable, de la cual derivan dos ramas taxonómicas principales: java.lang.Error y java.lang.Exception [1], [2].

La rama Error modela condiciones catastróficas del entorno de ejecución que se encuentran fuera del control de la aplicación, tales como el agotamiento de la memoria Heap (OutOfMemoryError) o el desbordamiento de la pila de llamadas (StackOverflowError) [1], [2]. Por norma de diseño, las aplicaciones no deben intentar capturar ni recuperarse de instancias pertenecientes a la jerarquía Error [1], [2].

Por el contrario, la rama Exception representa condiciones excepcionales recuperables producidas por la lógica de la aplicación o la interacción con recursos externos [2], [3]. Esta rama se subdivide conceptualmente en dos categorías:

1. Excepciones Verificadas (Checked Exceptions): Son subclases directas de Exception (excluyendo a RuntimeException) [2], [3]. El compilador exige de forma obligatoria que estas excepciones sean capturadas mediante bloques try-catch o declaradas en la firma del método con la cláusula throws [2], [3]. Ejemplos representativos incluyen IOException, FileNotFoundException y clases asociadas a servicios externos [2], [3].

2. Excepciones No Verificadas (Unchecked Exceptions): Coinciden con la jerarquía derivada de RuntimeException [2], [3]. Modelan fallas en la lógica del programador o violaciones de contratos de la API, tales como accesos fuera de rango en arreglos (IndexOutOfBoundsException), referencias nulas (NullPointerException), divisiones entre cero (ArithmeticException) o argumentos inválidos (IllegalArgumentException) [2], [3]. El principio de captura en superclases se fundamenta en la relación polimórfica ”es-un” (is-a) de la herencia [1], [3]. Dado que subclases como ArithmeticException o NullPointerException heredan de RuntimeException y esta a su vez de Exception, un bloque catch tiene la capacidad de interceptar cualquier instancia perteneciente a dicha familia taxonómica [1], [3].

3. DESARROLLO

3.1. Descripción de la Implementación (Captura en Superclases por Herencia)

El sistema desarrollado consta de los siguientes componentes modulares:

1. Clase Abstracta Base (ServicioBase):

o Define el método público ejecutar(), el cual contiene la estructura de control envolviendo la llamada al método abstracto de operación en un bloque try.

o Declara el método protegido abstracto #realizarOperacion(), cuya firma especifica la propagación potencial de fallas mediante throws Exception.

o Implementa el método protegido #capturarExcepcion(e: Exception), el cual actúa como el manejador centralizado en la superclase. Recibe polimórficamente la instancia de excepción y procesa la salida imprimiendo el nombre de la clase receptora vía e.getClass().getName() y el mensaje informativo vía e.getMessage().

2. Subclase Concreta (ProcesadorPago):

o Hereda de ServicioBase y sobreescribe (@override) el método #realizarOperacion().

o Implementa la regla de negocio de validación de saldo. Ante una condición de saldo insuficiente, instancia y lanza explícitamente una excepción no verificada mediante la instrucción throw new IllegalArgumentException(”Saldo insuficiente para procesar la transaccion.”).

3. Clase Principal de Ejecución (Main):

o Ubicada en el paquete mx.unam.fi.die.poo.g7.p10.Main, contiene el método estático main. Instancia la subclase ProcesadorPago asignándola a una referencia de tipo ServicioBase e invoca al método ejecutar().

3.2. Consultas a IA Generativa - Ejercicios 1.a y 1.b

- Ejercicio 1.a: Clases de la jerarquía Error en Java:

  o Integrante 1 (ChatGPT - OpenAI):

  ■ Prompt: "¿Cuáles son las clases principales dentro de la jerarquía de java.lang.Error en Java y por qué no se deben capturar?"

  ■ Respuesta IA: Detalló clases como VirtualMachineError, OutOfMemoryError, StackOverflowError y NoClassDefFoundError. Explicó que representan problemas graves de la JVM de los cuales la aplicación no puede recuperarse de forma segura.

  o Integrante 2 (Claude - Anthropic):

  ■ Prompt: "Explica el significado técnico de ClassNotFoundException frente a NoClassDefFoundError."

  ■ Respuesta IA: Indicó que ClassNotFoundException es una excepción verificada lanzada al intentar cargar clases dinámicamente, mientras que NoClassDefFoundError es un error crítico lanzado cuando el cargador de clases no encuentra una dependencia en tiempo de ejecución.

- Ejercicio 1.b: Excepciones más comunes en GUI:

  o Integrante 3 (Gemini - Google):

  ■ Prompt: "Consulta con IA cuáles son las excepciones más comunes al desarrollar interfaces gráficas de usuario (GUI) en Java Swing o JavaFX."

  ■ Respuesta IA: Listó NullPointerException al invocar métodos en componentes no inicializados, IllegalArgumentException al pasar parámetros inválidos a contenedores y IllegalStateException al intentar actualizar la interfaz desde un hilo secundario en lugar del Event Dispatch Thread.

  o Integrante 4 (Copilot - Microsoft):

  ■ Prompt: "¿Cómo afecta ClassCastException en la gestión de eventos de una GUI?"

  ■ Respuesta IA: Explicó que ocurre frecuentemente al realizar conversiones de tipo incorrectas sobre fuentes de eventos o modelos de tablas y listas.

  o Integrante 5 (DeepSeek):

  ■ Prompt: "Estrategias para prevenir fallos por excepciones no controladas en componentes gráficos."

  ■ Respuesta IA: Señaló el uso de validaciones defensivas previas al renderizado y la captura centralizada de excepciones en hilos de eventos.

3.3. Diagramas UML

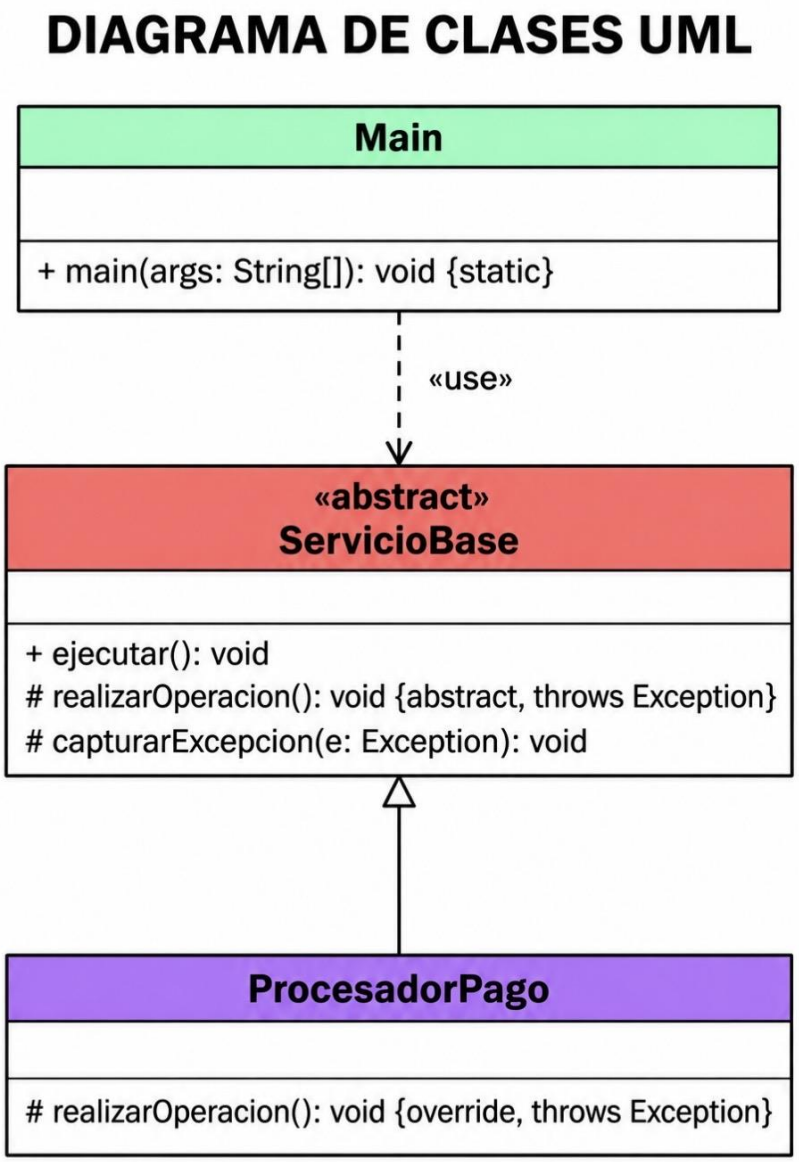


Figura 1: Diagrama de clases UML que ilustra la relación de herencia entre la superclase abstracta ServicioBase y la subclase concreta ProcesadorPago, especificando los métodos orientados a la captura polimórfica de excepciones.

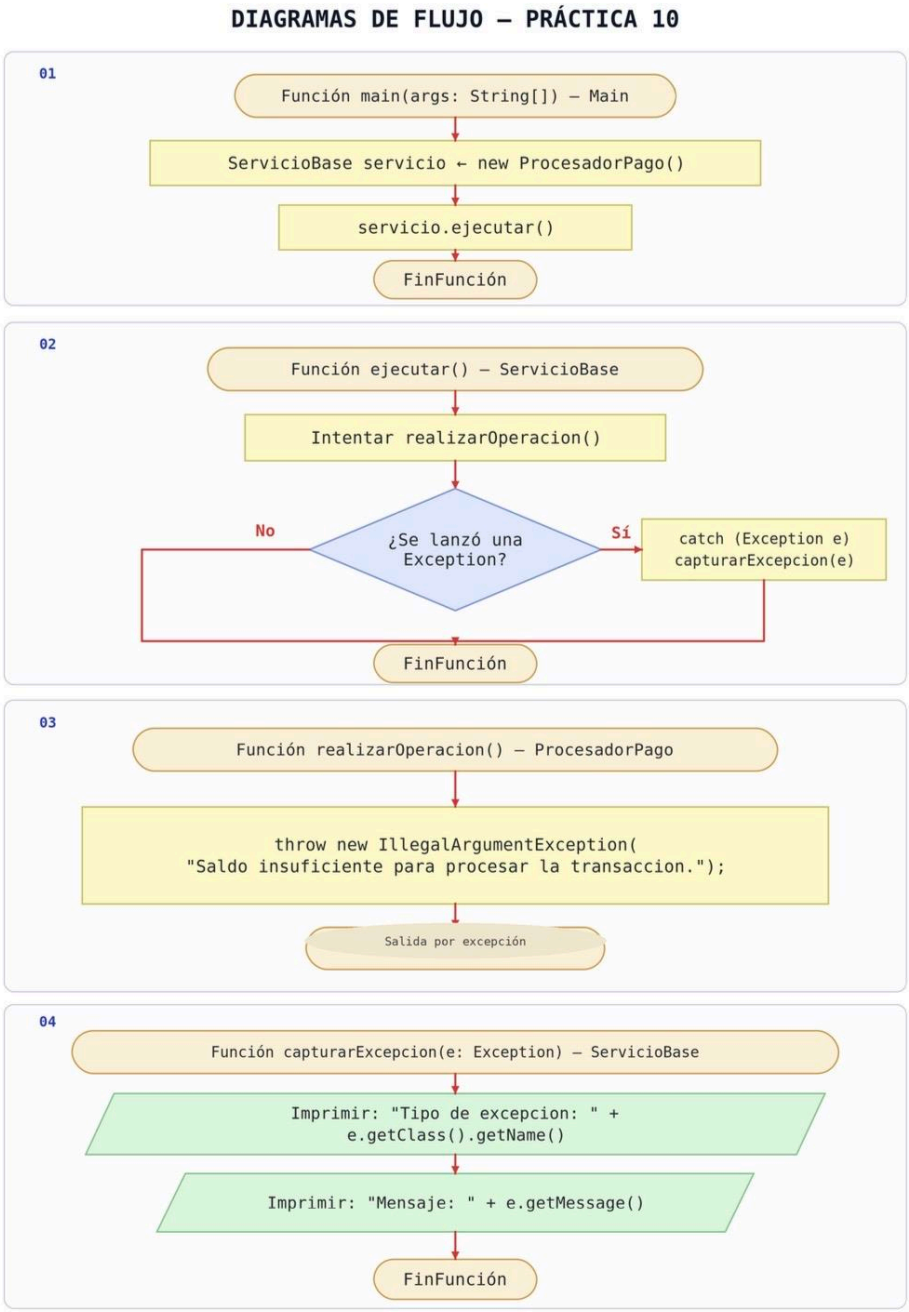


Figura 2: Diagramas de flujo de los módulos del sistema, detallando el control de ejecución secuencial, las estructuras de decisión y el bloque de captura centralizado ante el lanzamiento de la excepción.

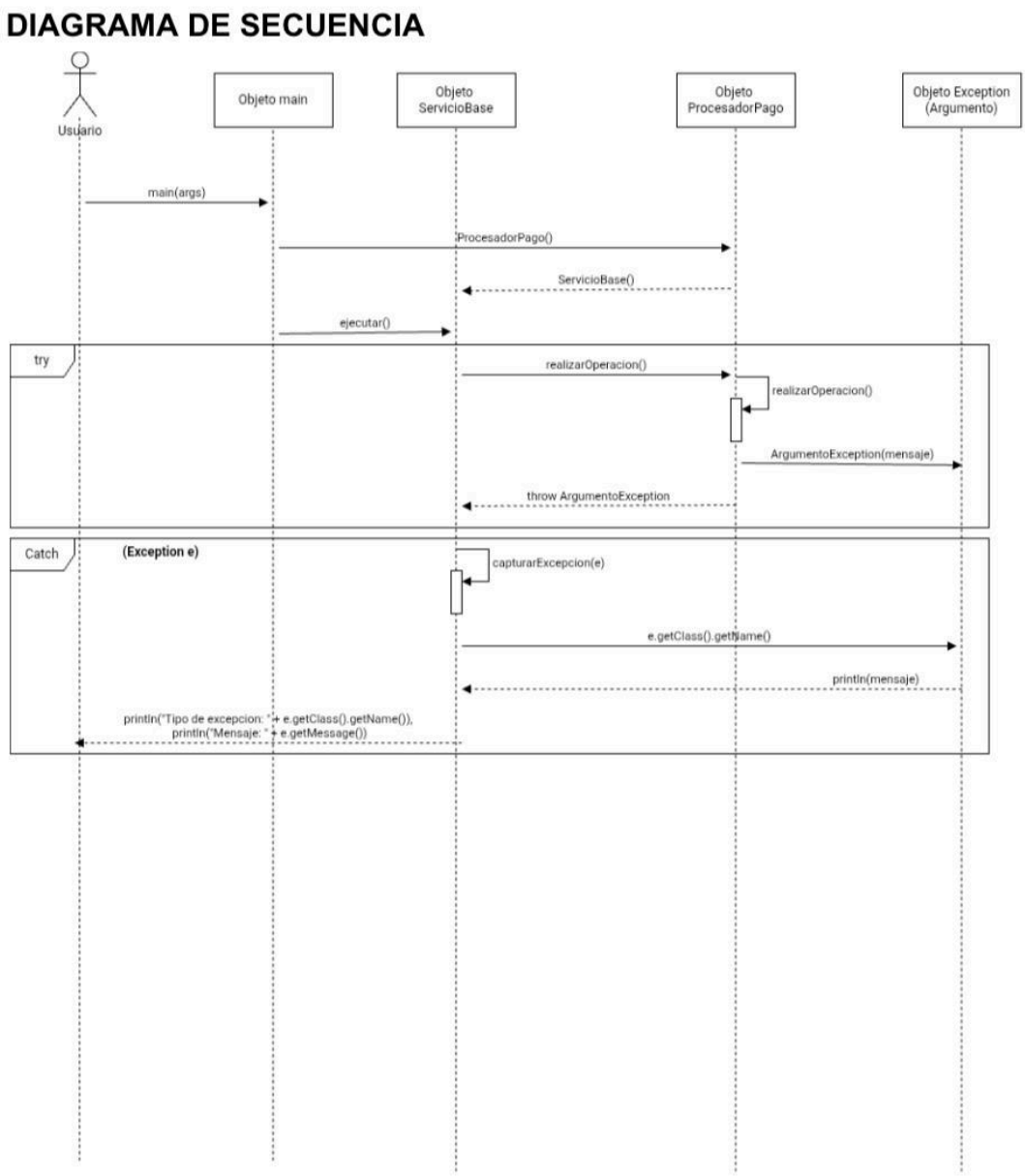


Figura 3: Diagrama de secuencia temporal que modela la interacción entre objetos y el flujo de llamadas envuelto en marcos try-catch para la gestión de errores en tiempo de ejecución.

3.4. Pruebas de Escritorio

- Prueba 1 (Flujo Nominal): Ingreso de datos válidos. El bloque try finaliza con éxito y se ejecuta el bloque finally liberando recursos.
- Prueba 2 (Captura de ArithmeticException en Superclase Exception): División entre cero. La JVM lanza la excepción y el bloque catch(Exception e) la intercepta polimórficamente, evitando el cierre abrupto del programa.
- Prueba 3 (Captura de Error de Formato): Conversión fallida de cadenas a números (NumberFormatException), capturada mediante la superclase base.
- Prueba 4 (Excepción Personalizada de Dominio): Disparo explícito de una excepción customizada heredada de Exception, verificando su correcta recepción en el manejador general.

4. RESULTADOS

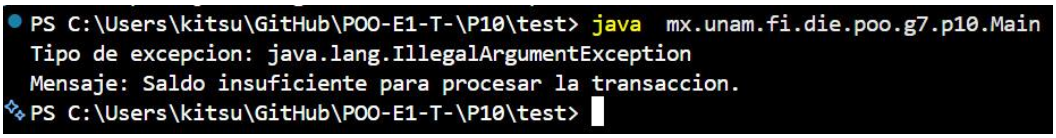


Figura 4. Captura de ejecución en consola del sistema mostrando la intercepción y manejo controlado de una excepción de tipo IllegalArgumentException, evidenciando el despliegue dinámico del tipo de error y su mensaje descriptivo asociado sin incurrir en una terminación abrupta del programa.

5. CONCLUSIONES

El desarrollo de la práctica permitió constatar que el manejo de excepciones mediante estructuras de herencia y polimorfismo constituye una herramienta fundamental para la construcción de sistemas tolerantes a fallos en el paradigma orientado a objetos. La demostración experimental del principio de captura en superclases evidenció cómo la abstracción taxonómica encauzada en la clase Exception permite procesar múltiples familias de fallas derivadas a través de un punto centralizado de gestión, simplificando la arquitectura del código sin sacrificar la capacidad de recuperación del sistema en tiempo de ejecución. Asimismo, el análisis diferenciador entre la jerarquía de errores irrecuperables del sistema y la jerarquía de excepciones de la aplicación permitió comprender la importancia de no interceptar la clase Error, garantizando que la máquina virtual gestione adecuadamente las situaciones críticas de infraestructura. La incorporación estratégica de bloques de captura polimórfica combinados con rutinas obligatorias de cierre en bloques finally asegura el mantenimiento de la consistencia del estado del software y evita la terminación intempestiva de las aplicaciones. En conclusión, el dominio sintáctico y conceptual de las excepciones verificadas y no verificadas en Java proporciona a los ingenieros en computación los fundamentos necesarios para diseñar aplicaciones robustas, mantenibles y preparadas para operar de forma segura bajo entornos de producción reales.

6. REFERENCIAS

[1] Oracle Corporation, "Exceptions," The Java Tutorials: Learning the Java Language - Essential Classes. [En línea]. Disponible en: https://docs.oracle.com/javase/tutorial/essential/exceptions/index.html

[2] Baeldung, "Exceptions in Java," Baeldung Computer Science & Java Core Tutorials. [En línea]. Disponible en: https://www.baeldung.com/java-exceptions

[3] GeeksforGeeks, "Exceptions in Java," GeeksforGeeks Computer Science Portal. [En línea]. Disponible en: https://www.geeksforgeeks.org/exceptions-in-java/

[4] Facultad de Ingeniería, Universidad Nacional Autónoma de México (FI UNAM), "Manual de Prácticas del Laboratorio de Programación Orientada a Objetos (Lineamientos LCP02)," Ciudad Universitaria, México: UNAM, 2026. [En línea]. Disponible en: http://lcp02.fi-b.unam.mx/